



Leading University

Computer Science & Engineering

Course Title: Computer Graphics Sessional
Course Code: CSE-4262
Project Title: Interactive 2D City Scene Simulation using Python OpenGL

Submitted To:

Name: Ashfakur Rahman
Designation: Adjunct Lecturer
Faculty: Computer Science & Engineering

Submitted By:

Name:	Samia Ferdous Proma
Department:	Computer Science & Engineering
Batch:	61
Section:	A
ID:	0182310012101006

Submission Date: 26-09-2026

PART 1: PROBLEM DISCOVERY

1. Which career track have you chosen?

- **Answer:** Track C: Game / Graphics Industry or Freelancing.

2. What is your project idea?

- **Answer:** An interactive 2D city scene simulation built using Python OpenGL. It features a dynamic urban environment including buildings, a roadway with traffic divider strips, an animated moving car driven by 2D transformations, a smart traffic light system controlling vehicle movement, roadside trees, the sun, and pedestrians along with children and pets walking on the sidewalk.

3. Which Computer Graphics concepts will your project use?

- **Answer:**
 - **Line & Shape Drawing:** Used to construct geometric structures such as buildings, roads, trees, and character blocks.
 - **Color Fill:** Used to apply custom RGB color fills across the sky, background, vehicles, and scenery elements.
 - **2D Transformations:** Utilizing translation matrix functions (`glTranslatef`) to animate the smooth horizontal movement of the car.
 - **Bezier Curves:** Implemented via Bernstein polynomials to render smooth curved vector paths.
 - **Line Clipping:** Demonstrated through viewport boundary clipping concepts.

4. Find two real-world systems using similar techniques.

- **Answer:**

- **System 1:** 2D Traffic Management and Signal Simulation Software utilized in urban planning to monitor traffic flow.
- **System 2:** 2D Side-scroller Game Engines (such as classic platformers and city-building games) that rely on 2D coordinate transformation and sprite rendering pipelines.

PART 3: REFLECTION REPORT

Q1. What were the primary technical challenges you faced during the implementation of the 2D City Scene, and how did you overcome them?

- **Answer:** The primary challenge was managing the animation loop alongside real-time state updates for the traffic lights and moving car without causing visual flickering. This was successfully resolved by utilizing double buffering (`glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB)`) and implementing a precise frame counter (`glutTimerFunc`) to handle smooth coordinate updates and continuous screen refreshing (`glutPostRedisplay`).

Q2. How do the 2D transformation and curve-drawing algorithms contribute to the interactivity and realism of the project?

- **Answer:** 2D transformations (specifically translation via `glTranslatef`) bring the static city scene to life by dynamically shifting coordinates over time, allowing the car to drive smoothly across the street. Furthermore, Bezier curves enable the rendering of organic, smooth pathways and curved road elements, breaking the monotony of rigid straight lines and enhancing the overall visual realism.

Q3. In what ways does integrating multiple graphic components (such as buildings, traffic lights, and pedestrians) test your understanding of coordinate geometry and viewport management?

- **Answer:** Combining diverse elements requires strict hierarchical structuring and precise local-to-world coordinate mapping. It tests the developer's capability to scale, translate, and position disparate objects (like tiny pedestrians versus large background buildings) within a unified Cartesian coordinate system without overlapping errors or boundary distortion.

Q4. What did you learn about graphics rendering pipelines and performance optimization while working with PyOpenGL?

- **Answer:** Working with PyOpenGL provided deep insights into immediate-mode and retained-mode rendering concepts. I learned that minimizing redundant state changes, optimizing vertex drawing calls, and regulating frame refresh rates are crucial for maintaining a stable 60 FPS performance, preventing CPU/GPU bottlenecks during real-time animation.

Q5. How can the concepts applied in this academic project be scaled or adapted for commercial game development or interactive simulation software?

- **Answer:** The fundamental principles applied here—such as vertex buffers, transformation matrices, state-driven animation loops, and event handling—form the bedrock of modern game engines like Unity or Unreal Engine. Scaled up with hardware acceleration, shader programming, and object-oriented architecture, these exact algorithms can power complex 2D indie games, urban traffic simulators, and interactive architectural walkthroughs.